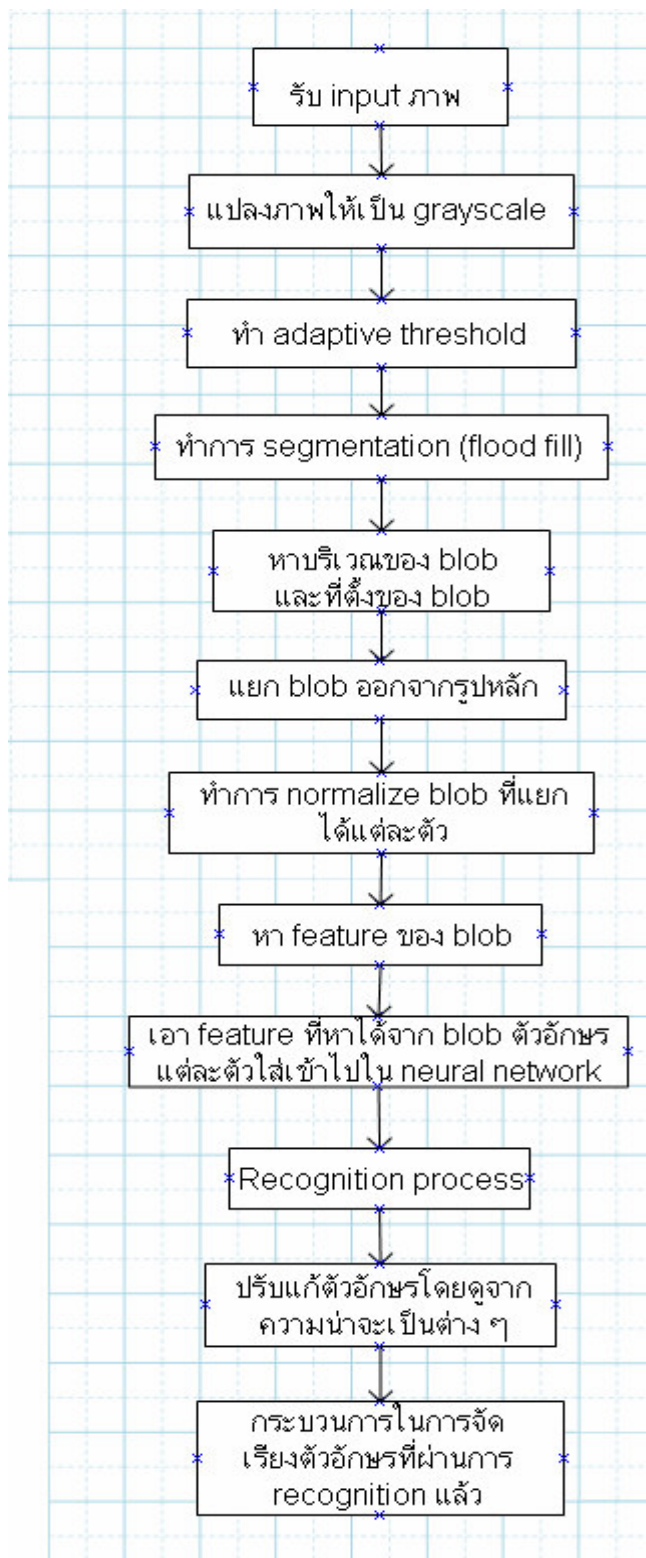


บทที่ 3

การออกแบบและพัฒนา

Block Diagram



Blob คือ บริเวณอิสระต่าง ๆ ในภาพที่เราสนใจ และสังเกตได้เพราะมันแยกกันอยู่ชัดเจน

ในบทนี้จะพูดถึงการออกแบบโครงงานซึ่งประกอบไปด้วยหลาย ๆ ส่วน ดังนี้

3.1 กระบวนการในการรับ input ภาพ

ใน project นี้ สิ่งสำคัญที่จำเป็นต้องมีก็คือ ส่วนของการรับภาพ ในการรับภาพของ project นี้เราได้อาศัย library เข้ามาช่วย เพื่อให้การจัดการเกี่ยวกับการ load ภาพ และ save ภาพ ทำได้สะดวกมากยิ่งขึ้น และ library ที่เราได้เลือกมาก็คือ opencv opencv นั้นเป็น library เกี่ยวกับ image processing ที่ open source และยังไม่ต้องเสียเงินอีกด้วย จุดเด่นของ opencv คือความง่ายในการใช้งาน ซึ่งทำให้เราสามารถเขียน code เพื่อทำการรับภาพ (load) หรือการ save ภาพได้เพียงแค่การเรียกใช้ function เพียง function เดียวเท่านั้น นอกจากนี้เรายังใช้ opencv ในการ show ภาพด้วย ต่อไปจะกล่าวถึงรูปแบบของ function ต่างๆใน opencv ที่ได้นำมาใช้ใน project นี้

การ load ภาพ

การ load ภาพทำได้โดยการเรียกใช้ function cvLoadImage ซึ่งมี prototype เป็นดังนี้

`IplImage* cvLoadImage(const char* filename,int iscolor = 1);`

Filename --> ชื่อของ file ที่จะถูก load

Iscolor --> รูปแบบสีเฉพาะของรูปภาพที่ถูก load ถ้ามากกว่า 0 รูปภาพที่ load จะโดนจัดให้เป็นรูปภาพ 3-channel ถ้าเท่ากับ 0 รูปภาพที่ load จะโดนจัดให้เป็น grayscale ถ้าน้อยกว่า 0 รูปภาพที่ load จะถูก load มาเป็น (ด้วยตัวเลขของ channel ขึ้นอยู่กับ file) ฟังก์ชัน cvLoadImage load รูปภาพจาก file จำเพาะ และ ส่งค่า pointer มาที่รูปภาพที่ load ในขณะนี้รูปแบบรูปภาพตามนี้ได้ถูกสนับสนุน

Windows bitmaps	- BMP, DIB;
JPEG files	- JPEG, JPG, JPE;
Portable Network Graphics	- PNG;
Portable image format	- PBM, PGM, PPM;
Sun rasters	- SR, RAS;
TIFF files	- TIFF, TIF.

การ save ภาพ

การ save ภาพทำได้โดยการเรียกใช้ function cvSaveImage ซึ่งมี prototype เป็นดังนี้

`Int cvSaveImage(const char* filename,const CuArr* image);`

Filename --> ชื่อของ file

Image --> รูปภาพที่ถูก save

ฟังก์ชัน `cvSaveImage` save รูปภาพไว้ที่ file จำเพาะ รูปแบบรูปภาพที่ถูกเลือกจะขึ้นอยู่กับ filename ที่ขยายออก มีเพียงรูปภาพ 8 bit single-channel หรือ 3-channel(ด้วย 'BGR' channel) ที่สามารถถูก save โดยใช้ฟังก์ชันนี้ ถ้ารูปแบบ,ความซับซ้อน หรือ ระดับ channel แตกต่างกัน ใช้ `cvCvtScale` และ `cvCvtColor` เปลี่ยนมันก่อนการ save หรือใช้ `cvSave` ที่มีโดยทั่วไป save รูปภาพในรูปแบบของ XML หรือ YAML

หลังจากที่เรา load ภาพหรือ save ภาพได้แล้ว ก็จำเป็นจะต้องมีการแสดงภาพด้วย ซึ่ง function ใน opencv ที่เกี่ยวข้องกับการแสดงภาพนั้นมีดังนี้

`Int cvNamedWindow(const char* name,int flags);`

function นี้เอาไว้ใช้สำหรับตั้งชื่อให้กับ window ที่จะเอามาแสดงภาพ และกำหนด property บาง อย่างให้กับ window โดยที่

Name ---> ชื่อของ Window ที่ซึ่งจะถูกใช้บ่งบอก window และจะปรากฏอยู่ที่ส่วนบรรยายใต้ window

Flags ---> Flags ของ Window ในขณะนี้จะมีเพียงตัวเดียวที่ยังสนับสนุน flag คือ `CV_WINDOW_AUTOSIZE` ถ้ามัน set ขนาดของ window จะปรับเปลี่ยนขนาดตัวเองโดยอัตโนมัติ เพื่อให้เหมาะสมกับการแสดงรูปภาพในขณะที่ user ไม่สามารถปรับเปลี่ยนขนาดเองได้ ฟังก์ชัน `cvNamedWindow` สร้าง window ที่ซึ่งสามารถใช้ placeholder สำหรับ images และ trackbars window ที่สร้างอ้างอิงโดยชื่อของมัน

ถ้า window มีชื่อของมันเองอยู่แล้ว ฟังก์ชันนี้จะไม่ทำอะไร

`Void cvShowImage( const char* name,const CvArr* image);`

function นี้เอาไว้สำหรับแสดง window ที่มีรูปภาพออกมาให้ user เห็น โดยที่

Name ---> ชื่อของ window

Image ---> รูปภาพที่จะทำการแสดง

ฟังก์ชัน `cvShowImage` แสดงรูปภาพใน window ที่จำเพาะ ถ้า window นั้นถูกสร้างขึ้นโดย `CV_WINDOW_AUTOSIZE` flag แล้ว รูปภาพจะถูกแสดงด้วยขนาดดั้งเดิม ถ้าไม่เช่นนั้น รูปภาพจะถูกสเกลให้เหมาะสมกับ window

เมื่อเราสามารถ load รูปภาพได้แล้ว ซึ่งรูปภาพที่เรา load เข้ามาจะต้องอยู่ในรูปแบบของ rgb 24 บิต (ประกอบด้วย r = 8 บิต, g = 8 บิต, b = 8 บิต)และเป็นภาพที่มี format เป็น bmp ด้วยขนาดของรูปจะมีขนาดเท่าไรก็ได้ หลังจากที่เราได้ภาพเข้ามาได้แล้ว เราก็จะทำการ casting รูปที่เข้ามาโดยเก็บรูปภาพโดยใช้เป็น pointer ของ char แทน เนื่องจากการเก็บข้อมูลของรูปภาพนั้น เมื่ออยู่ในหน่วยความจำมักจะเก็บเรียงกันไปเรื่อยเป็นแถวยาวซึ่งจะไม่ใช่เป็นอาเรย์ขนาด 2 มิติแน่นอนและเนื่องจากรูปภาพนั้นเก็บทีละ 8 บิตสำหรับ r, g, b ตามลำดับ ดังนั้นจึงมีความเหมาะสมแล้ว ที่เราจะ casting ไปเป็นอาเรย์ของ char (pointer ที่ใช้เป็นของ char) เพราะว่าข้อมูลที่เป็น char ก็มีขนาด 8 บิตด้วยเช่นกัน

เมื่อเรา casting รูปภาพให้อยู่ในรูปของอาเรย์ของ char ได้แล้วสิ่งที่เราต้องทำต่อไปคือเราต้องเก็บข้อมูลของภาพให้มองเป็น pixel เนื่องจากว่าเมื่อเรามองข้อมูลภาพให้เป็น pixel นั้นจะทำให้เราสามารถ process

ภาพได้ง่ายกว่าการมองภาพให้เห็นทีละ byte ทีละ byte ไปและเนื่องจากว่าภาพขนาด 24 บิตนั่นเอง ดังนั้น สิ่งต่อไปที่เราทำก็คือทำการ group อารีย์ของ char ข้างต้นเพื่อให้มองเป็น pixel ให้ได้โดย group ทีละ 3 byte (3 byte = 24 bit) การ group ที่ว่านี้ก็เหมือนกับการ casting โดยการ casting แบบนี้เราจะ cast โดยการ ใช้ struct แทนโดยใน struct นี้ประกอบไปด้วย component ของ blue green และ red ตามลำดับ เมื่อทำการ cast มาแล้วข้อมูลที่เป็นอารีย์ของ char ก็จะถูก match เข้าโดยตรงและถูกต้องกับแต่ละ component ได้ ใน ส่วนของ struct นี้เราตั้งชื่อมันว่า F_PIXEL โดยที่รูปร่างหน้าตาเป็นดังต่อไปนี้

typedef struct

```
{
    BYTE blue;
    BYTE green;
    BYTE red;
} F_PIXEL;
```

ลองสังเกตดูดี ๆ จากโครงสร้างที่อยู่ข้างบนนี้ จะพบว่าเราจะเอา blue มาก่อน แล้วตามด้วย green กับ red ตามลำดับ ที่เราต้องเอา blue มาก่อนก็เนื่องจากว่า ข้อมูลภาพที่เก็บอยู่ในหน่วยความจำนั้น มันจะเรียง เป็นดังนี้คือ bgrbgrbgr และจะเรียงแบบนี้ไปเรื่อย ๆ ดังนั้นเพื่อให้ component ของ blue , green , และ red มัน match กันได้อย่างถูกต้องกับโครงสร้างที่เราสร้างขึ้นมา เราจึงจำเป็นต้องเรียงเลขเช่นดังที่กล่าวมา ข้างต้นด้วย

3.2 การ threshold ใน project นี้

สำหรับใน project นี้ เราจะใช้การ threshold แบบที่เป็น adaptive เนื่องจากว่า การทำ adaptive threshold นั้นจะให้ผลของการ threshold ที่ดีกว่า global threshold เพราะภาพที่ได้จาก scanner นั้นเมื่อทำเป็น grayscale แล้วเราจะพบว่าในหลาย ๆ บริเวณของภาพ อาจจะมีการมีความแตกต่างของค่า grayscale ระหว่าง foreground กับ background ไม่เท่ากันก็ได้ ดังนั้นทางที่ดีจึงควรใช้การ threshold แบบ adaptive

เราจะใช้ adaptive threshold โดยการอนุญาตให้ผู้ใช้สามารถที่จะทำการกำหนดขนาดของ window เอง ได้ ทั้งนี้เพื่อให้ผู้ใช้ได้เห็นความแตกต่างของการกำหนดขนาดของ window ที่มีขนาดต่าง ๆ กัน ในที่นี้ขนาด ของ window ที่ผู้ใช้สามารถ กำหนดได้นั้นจะอนุญาตให้กำหนดได้เฉพาะเลขคี่เท่านั้น เป็น window ที่เป็น สี่เหลี่ยมจัตุรัสขนาด $n \times n$ (โดยที่ n เป็นเลขคี่) ที่ต้องทำเช่นนี้ก็เพื่อว่าเราจะสามารถหา pixel ที่เป็นจุด กึ่งกลางได้ง่ายกว่าการกำหนด window ให้เป็นแบบอื่น เมื่อเรากำหนด window ที่จะใช้ได้แล้ว ขั้นตอน ต่อไปที่ทำการคือ เอา window ที่ว่านี้ไปวางทาบบนภาพ โดยให้ window นี้อยู่ในภาพไม่เกินออกมาขอบ ของภาพด้วย เมื่อเราเอา window นี้ไปวาง ไว้ในบริเวณใดบริเวณหนึ่งของภาพแล้ว (ดูรูปข้างล่าง ประกอบด้วย) เราก็จะเอาค่า graylevel ของทุก ๆ pixel ที่อยู่ภายในขอบเขตของ window มาทำการบวก รวมกัน แล้วก็หารด้วยจำนวนของช่องทั้งหมดของ window ก็จะได้ค่าเฉลี่ยของค่า graylevel ทั้งหมดที่อยู่

ภายในบริเวณที่ window นั้นไปวางทับ และเราก็จะเก็บค่าเฉลี่ยที่ได้นี้เพื่อเอาไว้สำหรับเป็นค่า threshold สำหรับ pixel ทั้งหมดที่อยู่ภายใน บริเวณนั้นด้วย เมื่อเราทำการหาค่าของ threshold ของบริเวณนี้เสร็จแล้วต่อไปเราก็จะทำการเลื่อน window ที่ว่านี้ไปในบริเวณถัดไปโดยบริเวณถัดไปที่เราจะเลื่อน window ไปนั้นจะต้องเป็นบริเวณที่ยังไม่เคยถูกกำหนดค่า threshold มาก่อน พุดง่าย ๆ ก็คือเลื่อน window ไปโดยไม่ให้ window เดิมทับกับ window ใหม่ทับกันได้ จากนั้นเราก็จะเลื่อน window ไปเรื่อย ๆ เพื่อคำนวณค่าเฉลี่ยของ graylevel (threshold) จากซ้ายไปขวาและจากบนลงล่าง ไปจนกระทั่ง pixel ทุก ๆ pixel ในภาพได้ถูกกำหนดค่าที่จะเอาไว้ threshold เรียบร้อยแล้ว(ในที่นี้เก็บค่า threshold ไว้เป็นอาร์เรย์ขนาด 2 มิติ ที่มีขนาดเท่ากับ ความกว้าง x ความยาวของภาพที่รับเข้ามา) ต่อไปที่เราทำก็คือเราจะ apply ค่าของ threshold ที่อยู่ในอาร์เรย์นี้ให้กับค่าของข้อมูลภาพแต่ละ pixel ไปเลย ถ้าหากว่าค่า graylevel ของ pixel นั้น ๆ มีขนาดที่มากกว่าค่า threshold ที่ตรงกับ pixel ของมันก็จะถูกกำหนดให้ pixel นั้นเป็นสีขาว หากค่าของ graylevel ของ pixel นั้น ๆ มีขนาดที่น้อยกว่าค่า threshold สำหรับ pixel นั้น ก็จะถูกกำหนดให้ pixel นั้นเป็นสีดำ เมื่อเรา apply ค่า threshold กับทุก ๆ pixel ในภาพเราก็จะได้ภาพที่เป็นแบบ binary (ภาพที่มี 2 สี)

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

รูปที่ 3.1 ลำดับของ window ที่จะถูกใช้สำหรับ adaptive threshold

3.3 การ segmentation หรือการ floodfill

ในส่วนของการ flood fill นอกจากที่เราจะต้องระมัดระวังในเรื่องของ stack โดยหลีกเลี่ยงการเรียกใช้ฟังก์ชันแบบ recursive แล้ว ยังมีอีกสิ่งหนึ่งที่เรายังต้องระวังนั่นก็คือ ขอบของภาพ เมื่อมีจุด สีขาว ซึ่งเป็นจุดที่เราสนใจไปอยู่ในบริเวณขอบของภาพเลย ก็จะเกิดปัญหาคือ จุดสีขาวที่อยู่บน ขอบภาพนั้นจะไม่สามารถหาเพื่อนบ้านรอบทิศทั้ง 8 ด้านของมันได้ครบ ถ้าหากว่าเราไม่มีการป้องกัน ที่ดีก็อาจจะทำให้เกิดปัญหาของการอ้างตำแหน่งใน memory ที่ผิดพลาดได้ (เพราะอาจจะเกิดการอ้างอิง ไปในบริเวณของจุดที่ไม่มีอยู่ในภาพจริงก็ได้) เพื่อที่จะแก้ปัญหานี้ สิ่งที่เราทำก็คือ เราจะทำกำหนด

ขอบของภาพ ให้เป็นบริเวณที่จะทำการ flood fill ไม่ได้ เราทำเช่นนี้โดยการ fill ขอบของภาพ ทุกๆ ด้าน 4 ด้านให้เป็นสีดำ หรือสีของ background ก็จะสามารถที่จะแก้ปัญหาในลักษณะนี้ได้

โครงงานนี้จะใช้ stack เข้ามาช่วยในการ flood fill ด้วย โดยเราจะมอง stack เหมือนกับเป็นอาเรย์ ขนาด 1 มิติ และเราก็จะสร้าง image map ของภาพขึ้นมาด้วย image map ในที่นี้เอาไว้สำหรับ เป็นตัว mark เพื่อบ่งบอกถึงสถานะของจุด ว่า ได้ผ่านการ flood fill มาแล้วหรือยัง ถ้าจุดนั้นไม่ ถูกทำการ flood fill เลยก็จะกำหนดให้เป็น -1 ก่อน และหากว่าจุดนั้นผ่านการ flood fill มาแล้วเราก็จะ กำหนดให้มีค่าเป็น 0 เพื่อที่จะได้ไม่ต้องมา flood fill ซ้ำที่จุดเดิมอีกรอบ สำหรับ psuedo code ของการ flood fill โดยใช้ stack นั้นให้ดูได้จากบทที่ 2 ซึ่งได้กล่าวถึงการ flood fill มาบ้างแล้ว

เมื่อเราทำการ flood fill ไปนั้น ขณะที่เราทำการ flood fill เราก็จะทำการหาขอบเขตของ blob ไปด้วยในระหว่างที่ทำการ flood fill โดยขอบเขตที่วนนี้ก็จะได้แก่ xmin , ymin และ xmax , ymax (นั่นก็คือดูว่าใน blob นั้น จุดที่อยู่ในแกน x และ แกน y ที่น้อยที่สุดมีค่าเป็นเท่าไร และจุดที่อยู่ในแกน x และแกน y ที่มากที่สุดมีค่าเป็นเท่าไร) เนื่องจากว่ามีจุดอยู่ทั้งสิ้น 2 จุด ทำให้เราสามารถกำหนดบริเวณขอบ เขตของ blob ได้เป็นสี่เหลี่ยม บริเวณที่ตั้งและขอบเขตของ blob แต่ละ blob ที่หาได้นี้ถือว่ามีความสำคัญ เพราะเราจะใช้ค่าต่าง ๆ เหล่านี้ เพื่อใช้ในการ extract blob (หรือการแยก blob ออกมาจากภาพอีกที) ที่เราต้องทำการหาตำแหน่งขอบเขตของ blob ก่อนแล้วจึงค่อยมาทำการแยก blob ออกทีหลัง ก็เนื่อง จากว่า เมื่อเราทำการหาขอบเขตของ blob ไปพร้อมกับการแยก blob อาจจะมีข้อผิดพลาดขึ้นได้ เช่นลองดูตัวอย่างข้างล่างนี้

ฟ

รูปที่ 3.2 ตัวอักษรที่มีการ intersect กัน

จากรูปข้างบนเมื่อเราหาขอบเขตของ blob ทั้งสองตัว (สระ อี กับ ฟ) เราจะพบว่าบริเวณของ blob สองตัวนี้มีการ intersect กัน ซึ่งถ้าหากว่าเป็นเรามอง เวลาที่ทำการแยก blob เราก็จะเอา สระ อี ออกมาก่อน แล้วค่อยตามด้วย ฟ แต่ในทางคอมพิวเตอร์นั้นไม่ใช่ เพราะเมื่อเราทำการ process ภาพ เพื่อหา blob นั้นมันก็จะเจอ blob ของตัว ฟ ก่อนเนื่องจากว่า ฟ นั้นมันมีขอบบนที่อยู่สูงกว่า สระ อี นั้น เอง ดังนั้น ฟ จึงเป็น blob ตัวแรกเลย เมื่อเราหาขอบบนของ ฟ ได้แล้ว และทำการ extract blob ของ ตัว ฟ ออกมาจากรูปภาพ เราก็จะได้บางส่วน of สระ อี ติดมากับ blob ของ ฟ ด้วย ซึ่งก็เป็นสิ่ง ที่เรา ไม่ต้องการเลย (บริเวณของ ฟ นั้นไป intersect กับบริเวณของสระ อี) ดังนั้นเราจึงต้องเขียนอัลกอ ริทึม เพื่อจะต้องจัดลำดับก่อนหลังของ blob ที่จะถูก extract ออกมาจากภาพหลักอีกที เพื่อไม่ให้เกิดปัญหาดัง เช่นข้างต้นขึ้นมา

3.4 กระบวนการทางด้านการหาจำนวนของบรรทัดและระดับของตัวอักษร

ก่อนอื่นจะพูดถึง วิธีการที่จะหาระดับหลักให้ได้ก่อน ซึ่ง ระดับหลักนี้จำเป็นในการที่จะสามารถเอาไว้ตัดสินใจได้ว่าข้อความที่เราับเข้ามาประมวลผลนั้น มีอยู่ทั้งสิ้นกับบรรทัด ระดับหลักในที่นี้มีความหมายถึงระดับที่วางพยัญชนะ ซึ่งตามปกติแล้ว ข้อความที่เราับเข้ามาจะต้องประกอบ ด้วยพยัญชนะก่อน ถึงจะมีสระ หรือว่า วรรณยุกต์ตามมาได้ สำหรับวิธีการที่จะหาระดับหลัก ซึ่งเป็นระดับที่วางพยัญชนะนั้น สามารถทำได้ดังนี้คือ

สืบเนื่องมาจากการทำฮิสโตแกรมในแต่ละแถวของรูปนั้น ทำให้เราสามารถที่จะตัดสินใจหาช่วงที่มีความกว้างตามแกนตั้ง(y) ที่มากที่สุดได้ ซึ่งตำแหน่งล่างสุด ของช่วงที่กว้างที่สุดนี้ จะถือว่าเป็นระดับหลัก เนื่องจากว่า ช่วงที่กว้างที่สุดนั้น เป็นช่วงที่เอาไว้สำหรับวางพยัญชนะ ซึ่งโดยปกติแล้ว ข้อความในบรรทัดใด ๆ ก็จะต้องประกอบด้วยพยัญชนะอยู่แล้ว และพยัญชนะโดยส่วนใหญ่ก็จะมี ความสูงมากกว่า สระ และ วรรณยุกต์ ยกเว้นแต่ สระบางตัวอย่างเช่น สระ ใ ใ เป็นต้น ซึ่งจะมี ความสูง เท่ากับพยัญชนะเลย ซึ่งได้แก่ ตัว ป ฟ

นอกจากการหาระดับหลักจะใช้การเปรียบเทียบ ความกว้างในแต่ละชั้นแล้ว ยังเอาระยะห่างของแต่ละชั้นมาทำการคิดด้วย คือว่าระยะห่างระหว่างระดับต่าง ๆ ที่อยู่ ในบรรทัดเดียวกันนั้น จะมีค่าที่น้อยกว่าระดับที่ห่างกัน และอยู่คนละบรรทัด การทำเช่นนี้ได้ เราจะต้องกำหนดค่า threshold ขึ้นมาไว้ล่วงหน้าก่อน ซึ่งค่า threshold นี้จะเอาไว้เพื่อเป็นตัวบ่งบอกได้ว่า ระดับใด ๆ ก็ตามที่ได้มาจากการทำ histogram นั้นควรจะอยู่ในบรรทัดเดียวกันหรือไม่การหาค่า threshold นั้นเราสามารถทำได้ โดยดูจากตัวอย่างดังนี้คือ ถ้าหากว่าข้อความที่รับเข้ามาหลายบรรทัดเป็นดังนี้

กินข้าวกับส้มตำอยู่
เธอไปเรียนที่ไหนดมา

รูปที่ 3.3 ตัวอย่างของข้อความที่มีหลายบรรทัด

การกำหนดค่า threshold จะใช้ระยะห่างระหว่างสระ อี กับ ตัว ก เป็นสำคัญและเราจะหาว่า ถ้าหาก ระยะห่างระดับไหนที่มีความห่างมากกว่า threshold นี้ละก็ เราก็จะถือว่า ระดับสองระดับนั้นอยู่คนละบรรทัดกัน นอกจากนี้ถ้าเราสังเกตดี ๆ เราจะพบอีก ข้อจำกัดหนึ่ง ที่เป็นตัวช่วยในการตัดสินใจว่า ระดับไหนจะอยู่คนละบรรทัด นั่นคือ การดูจากลำดับความกว้างของระดับใด ๆ จากตัวอย่างข้างต้นเราจะพบว่าถ้ามองตามความกว้างของระดับใด ๆ ระดับแรกสุดที่เจอเลยคือ ระดับของ สระ อี ไม้โท ไม้หันอากาศ เป็น ต้น ระดับถัดมาก็

คือระดับของพิกเซล ในที่นี้เริ่มตั้งแต่ 0, 1 เป็นต้น ซึ่งมีค่าของความกว้างมากกว่า ระดับแรกแน่นอนอยู่แล้ว ระดับที่ 3 คือระดับของสระ อู ซึ่งพบว่ามีความกว้างที่น้อยกว่าระดับที่ 2 ดังนั้นเราจึงสามารถตั้งข้อกำหนดได้ว่า หากความกว้างของระดับลดลงจากเดิม ลงเหลืออยู่ประมาณ ครึ่งหนึ่งของความกว้างครึ่งก่อน และมีระยะห่างจากระดับครึ่งก่อนอยู่ในค่าที่เหมาะสม แสดงว่าระดับนี้ เป็น ระดับล่างสุดของบรรทัดนี้ แต่วิธีการนี้ก็มีข้อจำกัด เช่นกัน อย่างเช่นลองดูตัวอย่างต่อไปนี้

ก ก ก ก ก ก ก ก ก ก

ก ก ก ก ก ก ก ก ก ก

ก ก ก ก ก ก ก ก ก ก

รูปที่ 3.4 รูปตัวอย่างที่ในแต่ละบรรทัดนั้นมีความสูงของบรรทัดเท่ากัน

จากรูปข้างบนเราจะพบว่า ความกว้างของแต่ละระดับนั้นมีค่าที่เท่ากันและระยะห่างของแต่ละระดับก็มีค่าที่เท่ากันด้วยเช่นกัน ดังนั้นในกรณีนี้ค่า threshold ก็จะมีค่าที่เท่ากับ ระยะห่างของแต่ละระดับหักลบออกด้วยค่า ๆ หนึ่ง ที่น้อย ๆ เพื่อที่จะทำให้ใช้ค่านี้เพื่อเป็นค่า threshold ได้อย่างมีความถูกต้องเหมาะสม ต่อไปจะกล่าวถึงการพยายามที่จะหาระดับต่าง ๆ ในแต่ละบรรทัด

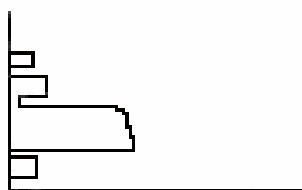
1. ก่อนอื่นเราจะทำการหา histogram สำหรับแต่ละแถวก่อน สำหรับกระบวนการ row histogram นั้นเราจะสามารถทำได้โดยการ ไล่จากแถวที่ 1 ไปจนถึงแถวสุดท้าย ซึ่งก็จะมีค่าเท่ากับ ความสูงของภาพ ในแต่ละแถวของภาพนั้น ให้เราทำการ scan เพื่อหาจุดของ pixel ที่เป็นสีขาว แล้วนับจำนวนจุดสีขาวนั้นที่อยู่ในแต่ละแถว จากนั้นก็เก็บในอาเรย์ไว้ กระบวนการนี้จะสามารถทำให้เราสามารถที่จะตัดสินใจว่าในรูปของเรานั้น น่าจะประกอบไปด้วยข้อความประมาณกี่บรรทัดได้

ปุมิกี่ขา

รูปที่ 3.5 รูปตั้งต้น

ปุมิกี่ขา

รูปที่ 3.6 รูปตั้งต้นที่ผ่านการ threshold



รูปที่ 3.7 ฮิสโทแกรม

2. เมื่อเราเสร็จสิ้นกระบวนการของ row histogram แล้ว สิ่งที่เราทำต่อไปก็คือ การหาความกว้างของช่วงต่าง ๆ ที่ปรากฏอยู่ใน histogram เช่น จากรูปตัวอย่างที่ปรากฏอยู่ข้างบนเราก็จะได้ row histogram ดังรูปด้านขวา นอกจากนี้เราจะหาความกว้างของช่วงที่ปรากฏใน histogram แล้ว เรายังหารระดับที่เป็นฐานของช่วงต่าง ๆ ด้วย (base line) เราจำเป็นที่จะต้องหา base line ในแต่ละช่วงของ histogram เพื่อที่จะเอาไว้สำหรับอ้างอิงถึงระดับต่าง ๆ ในแต่ละบรรทัด ทั้งนี้ก็เนื่องมาจากว่า ในภาษาไทยนั้น ในแต่ละบรรทัด นอกจากจะประกอบไปด้วยพยัญชนะแล้ว ยังประกอบไปด้วยสระ และ วรรณยุกต์ด้วย อย่างเช่นคำว่า "ปุมิกี่ขา" จากข้อความนี้ เราเลยสามารถที่จะจัดระดับในแต่ละบรรทัดของภาษาไทยได้เป็น 4 ระดับ ดังรูปด้านล่างนี้ กำหนดให้ สระ อู อยู่ในระดับล่างสุดหรือระดับที่ 1 ป อยู่ในระดับที่ 2 สระ อี อยู่ในระดับที่ 3 ไม้เอก อยู่ในระดับที่ 4 และเราทำการเก็บความกว้าง และระดับฐานของแต่ละช่วงไว้ใน อาร์เรย์ด้วย

ปุมิกี่ขา

รูปที่ 3.8 เส้นตรงในแนวนอนแสดงระดับต่าง ๆ ในบรรทัด

3. แต่เนื่องจากกระบวนการข้างต้น ยังไม่เพียงพอที่จะหาระดับได้อย่างถูกต้อง ในแต่ละบรรทัด เนื่องจากเมื่อเราสังเกตให้ดีในรูปข้างบนเราจะพบว่าช่วงของ histogram ช่วงที่ 2 นั้น ซึ่งมี ป เป็นตัวที่กำหนดความกว้างของช่วงนั้น แต่ว่ามีสระ อี ซึ่งในความเป็นจริงจะต้องถูกกำหนดให้เป็นอีกระดับหนึ่งด้วย เพราะว่ามันเป็นสระ แต่เนื่องจากว่า ความสูงของตัว ป นั้นไปทับกับระดับของสระ อี ด้วย ซึ่งทำให้ได้ผลลัพธ์ที่ไม่ถูกต้อง ดังนั้นทำให้เราต้องหา mechanism บางอย่างที่จะสามารถทำให้บอกได้ว่า ป กับ สระ อี นั้นมันอยู่คนละระดับกัน ถึงแม้ว่าทั้งสองตัวนี้จะอยู่ในช่วงของ histogram เดียวกัน

3.5 ปัญหาอันเนื่องมาจากกระบวนการ segmentation (flood fill)

ปัญหาที่เห็นได้ชัดคือ ปัญหาการ overlapped กันของ blob ตัวอย่างเช่น ข้อความที่ว่า “ป็นี่ปี่โก” จากการที่เราสังเกตข้อความนี้ เราจะพบว่ามี การซ้อนทับกันของ blob เกิดขึ้น อย่างเช่นคำว่า ‘ปี่’ เราพบว่าบริเวณ blob ของ สระอี กับ blob ของ ป นั้นมันทับกัน ดังนั้นเราจะต้องมี วิธีการ ในการแยก blob ของ สระอี กับ ป ออกจากกัน ซึ่งในที่นี้เราจะต้องแยก สระอี ออกมาก่อน เพราะว่ามันเป็นตัวที่ไปทับ ป จากนั้นก็จะสามารถแยก blob ป ออกมาได้ นอกจาก blob อาจจะซ้อนทับกันแบบโดยตรงๆซึ่งสามารถสังเกตได้อย่างชัดเจนแล้ว blob ยังสามารถซ้อนทับกันโดยทางอ้อมก็ได้ ในกรณีนี้จะหมายถึงว่า ระดับของ blob ตัวนั้นๆไปทับกับระดับของ blob ตัวอื่นๆ ทำให้ blob สองตัวนั้นมันมีระดับที่เหมือนกัน ทั้งๆที่เมื่อพิจารณาจากความเป็นจริง blob สองตัวนี้จะต้องมีระดับที่แตกต่างกัน ตัวอย่างเช่น จากข้อความที่ว่า “ฟางอยู่ในทุ่งนา” จากข้อความนี้ เราจะสังเกตได้เลยว่า blob แต่ละ blob ไม่มีการซ้อนทับกันโดยตรง แต่มีการซ้อนทับกันโดนอ้อม อย่างเช่น

ตัว ฟ กับ ไม้เอก จะถูกจัดให้อยู่ในระดับเดียวกันเพราะขอบเขตของ blob ของทั้งสองตัวนั้นซ้อนทับกันแกน y ดังนั้นเราจึงต้องมี วิธีการ เพื่อที่จะบอกให้ได้ว่า ความจริงแล้ว ฟ กับ ไม้เอก อยู่ในระดับที่ต่างกัน

ต่อไปจะกล่าวถึง วิธีการ ที่ใช้จัดการกับ blob ที่มีการซ้อนทับกันเกิดขึ้นไม่ว่าจะโดยทางตรงหรือทางอ้อมก็ตาม

การซ้อนทับกันโดยทางตรงจะสามารถจัดการได้ดังนี้



รูปที่ 3.9 ตัวอย่างของการทับกันโดยตรง

ในการที่จะตัดสินใจว่า blob ใดๆมีการ intersect เกิดขึ้นหรือไม่นั้นเราจะใช้วิธีการทำโดยตรวจสอบดูว่าขอบเขตบริเวณที่ blob ตัวนั้นๆอยู่ มีการไปทับกับขอบเขตบริเวณของ blob ตัวอื่นๆหรือไม่ วิธีการหนึ่งที่ได้ได้โดยการสร้าง array ขนาด 2 มิติขึ้นมา จากนั้นก็เติม array 2 มิตินั้นด้วยหมายเลข blob ของตัวอักษร ถ้าหากว่า บริเวณในภาพนั้นไม่มี blob ตั้งอยู่ ให้กำหนดค่าเป็น -1 ไว้ แต่ถ้าหากว่าเป็นบริเวณที่มี blob วางอยู่ ให้กำหนดค่าใน array 2 มิตินั้นให้เป็นหมายเลขของ blob ด้วย

เราจะพบว่าในขอบเขต blob ของตัวอักษร ป นั้น นอกจากจะมีเลข blob ของตัว ป แล้ว ยังมีเลข blob ของตัว สระอ อยู่ภายในบริเวณนั้นด้วย ทำให้บ่งชี้ได้ว่าการ intersect กันเกิดขึ้นแน่นอน และเราก็จะทำการเก็บข้อมูลไว้ด้วยว่า blob ตัวไหนมีการ intersect กับ blob ตัวไหนอีกด้วย

ต่อไป เราก็จะเอา blob ตัวที่มันไปทับกับตัวอื่น ๆ มาดูว่ามันไปทับกับตัวไหน สังเกตได้ว่าการทับกันของ blob ข้างต้นนั้น ในความเป็นจริงแล้วจะต้องทำให้เกิดระดับใหม่ขึ้นมาด้วย เนื่องจากว่าตัวสระ อ กับตัว ป นั้น มันควรที่จะอยู่คนละระดับกัน เราสามารถที่จะหาระดับใหม่ได้ โดยการเปรียบเทียบขนาดของ blob 2 อันใด ๆ ที่มัน intersect กันเราจะพบว่า blob ที่มีขนาดที่น้อยกว่านั้น จะทำให้เกิดระดับใหม่ขึ้นมา นอกจากจะใช้ในการตัดสินใจระดับได้แล้ว การหาขนาดของ blob ที่ intersect กัน 2 อัน ก็สามารถที่จะช่วยในการที่จะแยก blob ของตัว ป กับ สระ อ มาได้โดยมีลำดับที่ถูกต้อง ลองคิดดูว่าคำว่า "ปี" หากว่าเราเอาตัว ป ออกมาก่อนก็จะทำให้ blob ที่แยกออกมามีบางส่วนของสระ อ ติดมาด้วยซึ่งทำให้เกิดความผิดพลาดได้ ซึ่งการแยกที่ถูกต้องนั้นจะต้องเอาสระ อ ออกมาก่อนถึงค่อยเอา ป ออกมา หรือหมายความว่าอีกนัยหนึ่งก็คือ เราจะแยกเอา blob ที่มีพื้นที่น้อยกว่าออกมาก่อน blob ที่มีพื้นที่มากกว่า แต่การทำแบบนี้ก็ไม่ว่าจะถูกต้องเสมอไปหรือ

การซ้อนทับกันโดยทางอ้อม จะสามารถที่จะจัดการแก้ไขได้ดังนี้

เมื่อเกิดปัญหาการซ้อนทับกันโดยทางอ้อม ดังที่เคยกล่าวไว้แล้วใน paragraph ก่อน ๆ เพื่อที่จะแก้ปัญหา
นี้ ผู้พัฒนาได้ใช้วิธีการดังนี้ คือ

1. จะต้องหาระดับหลักของแต่ละบรรทัดให้ได้ก่อน ระดับหลักก็คือระดับที่เอาไว้สำหรับวางพยัญชนะ
2. เมื่อทำการหาระดับหลักได้แล้ว ก็จะต้องดูว่า ในระดับหลัก หรือ ระดับที่เอาไว้สำหรับวางพยัญชนะนี้ มีความกว้างเป็นเท่าไรด้วย เพื่อที่จะเอาไว้กำหนดขอบเขตบนของมันได้
3. เมื่อเราทำการหาความกว้างของระดับหลักได้แล้ว เราก็จะดูว่ามี blob ตัวไหนที่ระดับล่างของมันอยู่
ระหว่างขอบล่างของระดับหลัก แต่ไม่เกินขอบบนสุดของระดับหลัก และ blob ตัวนี้จะต้องมีระดับล่างสุด
ไม่เท่ากับระดับล่างสุดของระดับหลักด้วย
4. ถ้าหากว่ามี blob ดังที่กล่าวถึงในข้อที่ 3 นั้น เราก็จะเอาระดับล่างของ blob ตัวนั้นมาจัดให้เป็นระดับ
ใหม่ได้อีกระดับหนึ่ง

การ intersect กันโดยทางอ้อมอีกรูปแบบหนึ่งที่เจอ จะมีลักษณะดังรูปด้านล่างนี้

๐๐๐๐๐๐๐

รูปที่ 3.10 การซ้อนทับกันโดยทางอ้อมกับระดับ 0

เมื่อเราสังเกตจากรูป เราจะพบว่า ๐ นั้นมีการ intersect กันโดยทางอ้อมกับสระ อ สำหรับ การแก้ปัญหา
นี้ เราจะไม่สามารถแก้ไขโดยใช้วิธีการเดียวกับที่กล่าวมาข้างต้นได้ซึ่งวิธีการข้างต้นนั้นเอาไว้ สำหรับแก้ปัญหา
เมื่อ blob ตัวที่มัน intersect กันโดยทางอ้อมนั้น อยู่สูงกว่าขอบล่างสุดของระดับหลักแค่ นั้น ดังนั้นเรา
จำเป็นต้องหาวิธีการอื่น

ถ้าหากว่าเราใช้ algorithm ดังที่ได้กล่าวมาข้างต้นกับภาพที่อยู่ด้านบนนั้น จะทำให้เราไม่สามารถที่จะ
แยกแยะระหว่างระดับหลัก กับระดับของสระ อ ได้ เพราะว่ามันทับกัน (ซึ่งในความเป็นจริงระดับหลักจะถูก
กำหนดให้เป็นระดับที่ 1 และ สระ อ นั้นจะถูกกำหนดให้เป็นระดับที่ 0)

วิธีการแก้ไขเราจะใช้ วิธีการหา blob ของตัวที่มีลักษณะเป็นสระ อ หรือ สระ อุ กล่าวคือ เราจะใช้ความสูง
ของ blob เข้ามาเกี่ยวข้องด้วย ถ้าหากว่า blob ตัวนั้น มีขอบล่าง อยู่บนขอบล่างของระดับหลัก และก็มีความ
สูงของ blob น้อยกว่า ครึ่งหนึ่งของความกว้างของระดับหลักก็จะ แสดงว่าจะต้องมีสระประเภท สระ อ สระ
อุ อยู่ด้วย ถ้าหากว่า ยังไม่มีระดับที่ 0 ก็ให้เรากำหนดให้มีระดับที่ 0 เพิ่มเข้ามาด้วย

นอกจากนี้ปัญหาอีกปัญหานึง ที่เป็นที่น่าปวดหัวสำหรับเราเลยก็คือ ปัญหาของสระ อะ นั้นเอง
เนื่องจากว่าสระ อะ นั้น ความเป็นจริงควรที่จะถูกจัดให้อยู่ในระดับเดียวกันกับพยัญชนะ แต่จากลักษณะ

รวมทั้งตำแหน่งที่มันอยู่จึงทำให้หลายครั้งมีความผิดพลาดเกิดขึ้นจากกระบวนการ segmentation โดยเฉพาะอย่างยิ่งมีส่วนทำให้ ตำแหน่งของระดับต่าง ๆ ผิดเพี้ยนไปลอง สังเกตดูจากตัวอย่าง

ครูกินปะหมี่อยู่

รูปที่ 3.11 บรรทัดที่มีสระอะปนอยู่ด้วย ทำให้ได้ระดับทั้งหมดเพิ่มเป็น 5 ระดับ

จากข้อความข้างต้น เราจะสามารถหาระดับได้ถึง 5 ระดับ ซึ่งมันไม่ถูกต้องสำหรับข้อความภาษาไทย ที่จะมีอย่างมากได้แค่ 4 ระดับเท่านั้น ดังนั้นเราจำเป็นต้องมีวิธีการในการที่จะกำจัดสระ อะ ออกไปเพื่อไม่ให้มันถูกจัดไว้ว่าเป็นอีกระดับหนึ่งในบรรทัด หรือทำให้รู้ว่าสระ อะ นั้นอยู่ในระดับเดียวกันกับระดับของพยัญชนะเป็นต้น ซึ่งในวิธีการที่คิดขึ้นไว้สำหรับจัดการกับสระ อะ นี้ เป็นดังนี้

เนื่องจากว่าสระ อะ นั้นประกอบไปด้วย blob จำนวน 2 อัน และ blob ตัวที่อยู่ด้านล่างนั้น มันก็จะติดกับระดับหลักในแต่ละบรรทัดนั้น ๆ ด้วย เมื่อเราจะทำการ check สระ อะ เราจำเป็นต้องทราบ ความกว้างของบรรทัดนั้น ๆ ก่อน จากนั้น เราก็จะทำการสุ่ม blob ของตัวอักษร ที่อยู่ในบรรทัดนั้น ๆ ขึ้นมา โดยที่ blob ที่สุ่มขึ้นมาจะต้องมีขอบล่าง เป็นค่าเดียวกับขอบล่างของระดับหลักด้วย จากนั้นเราก็จะทำการหาอัตราส่วนของความกว้างของระดับหลัก ต่อ ความกว้างของตัวอักษรหนึ่งตัวในระดับหลัก การทำเช่นนี้มีข้อดีอีกอย่างคือว่า จะสามารถทำให้รู้ได้ว่า ในบรรทัดนั้น ๆ มีตัวอักษรที่พิเศษประกอบอยู่ด้วยหรือไม่ (อักษรพิเศษคือพยัญชนะที่สูงผิดปกติอย่างเช่น 'ป' 'ฟ' 'ผ' เป็นต้น) อัตราส่วนของความสูงของตัวอักษรที่พิเศษ ต่อความกว้างของมัน จะมีค่าที่มากกว่า อัตราส่วนนี้ของตัวอักษรแบบธรรมดา (ตัวอักษรแบบธรรมดาอย่างเช่น 'ก' 'ข' 'บ' 'อ' เป็นต้น) เพราะเราสังเกตได้ว่าการมีสระ อะ นั้น เนื่องจากถ้ามองกันดี ๆ แล้วจะพบว่ามันก็เป็นปัญหาของการซ้อนทับกันโดยทางอ้อมอีก แต่คราวนี้สระ อะ ไม่ควรจะถูกจัดให้เป็น ระดับใหม่ในบรรทัดนั้น ๆ การซ้อนทับกันโดยทางอ้อมนั้นส่วนใหญ่เกิดขึ้นกับตัวอักษรแบบพิเศษ เพราะว่าตัวอักษรแบบธรรมดานั้นเมื่อเกิดการซ้อนทับกันโดยทาง อ้อมก็ไม่เกิดปัญหาอะไรขึ้นมาเลย ลองสังเกตดูจากรูปตัวอย่างด้านล่าง

กระป๋องมังกร

รูปที่ 3.12 รูปนี้สระอะจะไม่มีผลกับกระบวนการหาระดับ

ถ้าเป็นเช่นรูปข้างบน ก็ไม่จำเป็นจะต้องเอาไปเข้าอัลกอริทึม เพื่อหาการซ้อนทับกันโดยทางอ้อม เพราะว่าในกรณีนี้ สระ อะ มันก็จะอยู่ในระดับหลักของบรรทัดถูกต้องอยู่แล้ว แต่ถ้าหากว่าเป็นดังรูปข้างล่างนี้

ฝึกทำความเข้าใจ

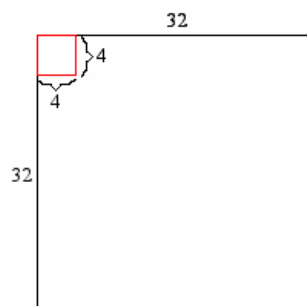
รูปที่ 3.13 บรรทัดที่มีทั้งสระอะ และตัวอักษรพิเศษ

เมื่อมีตัวอักษรพิเศษปรากฏขึ้นมา ในที่นี้ได้แก่ ๘ ปรากฏขึ้นในบรรทัดใด ๆ ควรที่เราจะเอาไปเข้าอัลกอริทึมเพื่อหาการซ้อนทับกันโดยทางอ้อมด้วย เพราะว่าอาจจะมีสระ อี อี เป็นต้น ถูกบังโดยตัว ๘ อยู่ได้ ถ้าหากว่าเรา หาระดับใหม่จากรูปข้างบน โดยไม่ได้กำหนดข้อจำกัดอะไรเลย เราก็จะได้ระดับใหม่เพิ่มขึ้นมาอีก 2 ระดับคือ ระดับของสระ อะ ตัวบน กับระดับของสระ อี แต่สระ อะ ไม่ควรที่จะเป็นระดับใหม่ แต่จากการสังเกตดู เราจะพบว่าขอบเขตล่างของ blob สระ อะ ตัวบนนั้นอยู่ต่ำกว่า ขอบเขตล่างของ blob สระ อี และ ขอบเขตล่างของ blob สระ อะ นั้น ก็จะอยู่สูงจากระดับหลักในบรรทัดนั้น ๆ น้อยกว่า 6/10 ของความกว้างของระดับหลัก ในขณะที่ ตัว สระ อี นั้น ขอบเขตล่างของมัน อยู่สูงจากระดับหลักมากกว่า 6/10 ของความกว้างของระดับหลัก โดยการใช้อนุกรมที่ได้อ้างมานี้ จะสามารถทำให้เราสามารถที่จะกำจัด สระ อะ ไม่ให้มันไปเป็นอีกระดับหนึ่งได้

เมื่อเราสามารถที่จะทำการหาระดับต่าง ๆ ที่อยู่ในแต่ละบรรทัดได้แล้ว รวมไปถึงจำนวนของบรรทัดทั้งหมดที่อยู่ในรูปภาพที่เราับเข้ามาเพื่อจะแปลงเป็นตัวอักษรแล้วละก็ ขั้นตอนต่อไปที่เราทำก็คือการแยกเอา blob ของตัวอักษร ทั้งหมดที่อยู่ในรูปภาพออกมาให้ได้ เพื่อที่จะเอา blob ที่ว่านี้มาคุณลักษณะ และเอา blob ไปเข้าสู่กระบวนการ recognition เพื่อที่จะสามารถบ่งบอกได้ว่า blob ตัวนั้น ๆ จะหมายถึงพยัญชนะตัวใดในภาษาไทย สำหรับกระบวนการแยก blob แต่ละตัวออกมาจากภาพนี้ เราจะไม่ใช่กระบวนการของ row histogram เลย แต่เราจะอาศัยวิธีการที่เรียกกันว่า flood fill เพื่อที่จะมาหาบริเวณของเขตของ blob แทน ส่วนในเรื่องของขั้นตอนวิธีการของการ flood fill นั้น ได้กล่าวมาก่อนแล้วในช่วงต้น ๆ ของรายงานชิ้นนี้ การ flood fill สำหรับการหา blob นี้เราจะใช้การ flood fill แบบ 8 ทิศทาง เพื่อที่จะเข้าถึงจุดทุกจุดที่เชื่อมกันของ blob ได้ และเราจะ flood fill เฉพาะภาพที่ผ่านการ threshold มาแล้วเท่านั้น แต่ก็มีสิ่งที่จะต้องระวังเช่นกัน สำหรับการ flood fill กล่าวคือ เมื่อเราทำการ threshold ภาพที่รับเข้ามา ให้กลายเป็นภาพแบบ binary แล้ว การทำ threshold นั้นจะต้องระมัดระวัง เพื่อให้เมื่อทำการ threshold แล้ว ต้องทำให้ไม่มี blob ของตัวอักษรใด ๆ ติดกันเลย เพราะถ้าหากว่า มีตัวอักษรที่ติดกันอยู่ เมื่อเราทำการ flood fill ก็จะได้ภาพที่มีตัวอักษรอยู่ 2 ตัว ทำให้ไม่สามารถที่จะเอาไปทำการ recognition ได้ เพราะเราทำการ recognition โดยใช้ blob ของตัวอักษรเพียง 1 ตัวเท่านั้น ในการแยก blob นั้นเราก็จะใช้การ flood fill ให้กับทุก ๆ blob ที่อยู่ในภาพจน

ครบทุก blob เมื่อ แยก blob ตัวไหนออกมาได้แล้วก็จะเติม blob ที่ทำการแยกเสร็จแล้ว จากสีขาว ให้กลายเป็นสีดำ เพื่อที่จะเป็นการ mark ว่าไม่ต้องมา flood fill blob ตัวนี้อีกแล้ว

เมื่อเราทำการแยก blob ออกมาแล้ว เนื่องจากว่า blob ที่แยกออกมานั้น อาจจะมีขนาดที่ต่างกันได้ และมีขนาดที่ต่างจากข้อมูลที่เราไป train เพื่อเอาไว้สำหรับการ recognition ดังนั้นเราจึงจำเป็นต้องหา feature ที่มีลักษณะเด่นเพื่อเอาไว้สำหรับเป็นตัวแทนของ blob ได้ สิ่งแรกที่เราทำก็คือ จะทำการ scale blob ของตัวอักษรให้มีขนาดที่เป็นมาตรฐาน ขนาดหนึ่งๆก่อน ซึ่งไม่ว่า blob ที่ได้จากระบวนการ flood fill จะมีขนาดเป็นเท่าใดก็ตามก็จะถูก scale ให้อยู่ในขนาดมาตรฐานเท่ากันหมดเลย ในที่นี้ทางผู้จัดทำได้ ใช้การ scale blob ใดๆให้มีขนาดเป็น 32x32 pixel ส่วนในเรื่องของการ scale นั้นได้เลือกใช้ algorithm ในการ scale แบบ bilinear หรือ bilinear interpolation ซึ่งก็ได้เคยกล่าวไว้แล้วในหัวข้อก่อนหน้านี้แต่จากการ scale นั้นเราจะพบว่า เมื่อเรา scale จากภาพ (blob) ที่มีแค่ 2 สีคือสีขาวกับสีดำเท่านั้นให้ได้ขนาดที่ใหญ่ขึ้นไปเป็น 32x32 pixel นั้นเราจะพบว่าเมื่อทำการ scale blob เสร็จแล้วกลับไม่ได้ภาพที่เป็น 2 สีแต่กลายเป็นภาพแบบ grayscale แทนและก็มีจำนวนสีในแต่ละ blob ไม่คงที่แตกต่างกันไป ซึ่งลักษณะเช่นนี้ก็เป็นข้อดีอีกอย่างหนึ่งเพราะว่า เมื่อข้อมูลของ blob มีความแตกต่างกันมากขนาดนั้นก็จะทำให้กระบวนการแยกนั้นสามารถที่จะมีความถูกต้องมากกว่าเมื่อข้อมูลของ blob มีความแตกต่างกันน้อยๆ (พูดได้ง่ายๆก็คือว่ามี feature ที่มากกว่า) เมื่อเราทำการ scale blob ไปเป็นขนาด 32x32 pixel เสร็จเรียบร้อยแล้ว เนื่องจากว่ามีข้อมูลเกี่ยวกับ blob มากเกินไป เพราะถ้าเราจะหาข้อมูลนี้ไปใช้ เราจะต้องเอา memory สำหรับแต่ละ blob เป็นขนาดถึง 1024 ซึ่งถ้ามี blob จำนวนมากที่จะต้องเอาไป train หรือเอาไป recognition นั้น ก็จะทำให้ใช้เวลาในการ PROCESS นานพอสมควร ตัวนั้นเพื่อความเหมาะสม หลังจากที่เราทำการ scale blob ไปเป็น 32x32 แล้วเราก็จะทำการ group ให้ได้เป็นขนาด 8x8 โดยการ group นั้นจะทำได้ดังนี้ สังเกตจากรูป



รูปที่ 3.14 สีเหลี่ยมสีแดงคือ feature 1 อันที่เราหาได้จาก blob ขนาด 32 x 32

เมื่อเราสร้าง window จตุรัสขนาด 4x4 แล้วเอา window นี้ไปใส่ไว้ใน blob ที่มีขนาดเป็น 32x32 เราจะสามารถวาง window ขนาด 4x4 นี้ได้เต็มบริเวณของ blob นั้นทันที และขนาดของ window ที่วางนี้ก็จะมีความเท่ากับ 8x8 หรือ 64 สำหรับกระบวนการหา feature นั้นเราก็จะทำตามนี้นั่นเอง จากนั้นเราก็จะเอาค่าที่อยู่ใน blob ขนาด 32x32 ที่อยู่ในแต่ละ window ซึ่งมีขนาดเท่ากับ 16 ช่องมาทำการบวกรวมกัน

ให้หมดครบทุกๆช่องในแต่ละ window จากนั้นก็จับหารด้วย 16×255 ซึ่งค่า (16×255) นี้ก็จะหมายถึงว่า window นั้น ๆ มีสีขาวหมดเลยเพราะว่าค่า 255 ในทาง grayscale นั้นก็หมายถึงสีขาว และที่ 16 ที่เอามาคูณกับ 255 ก็ถือว่าเป็นจำนวน ช่องทั้งหมดที่อยู่ใน window สุดท้ายก็จะเอาค่าผลบวกทั้งหมดที่ได้ตอนต้น มาทำการหารด้วยค่า 16×255 เพื่อเป็นการทำให้ค่าของ window ขนาด 16×16 นั้นมีค่าที่เป็น ทศนิยม และมีค่าอยู่ในช่วง $0.0 - 1.0$ ซึ่งจากการที่มีจำนวนของ window ทั้งหมด 8×8 ก็จะทำให้เราได้ ค่าที่เป็นทศนิยม ตั้งแต่ $0.0 - 1.0$ จำนวนทั้งสิ้น 64 ค่า และเราก็จะใช้ค่านี้เพื่อเป็นตัวแทนของ blob ตัวอักษรแต่ละตัว ต่อไป จากนั้นเราก็จะใช้ค่าทั้ง 64 ค่าที่หาได้นี้ รวมกับอัตราส่วนของความสูงของ blob ต่อความกว้างของ blob ซึ่งก็จะได้ค่าออกมาทั้งสิ้น 65 ค่า เพื่อเอาไว้สำหรับ recognition ว่า blob ตัวนั้น ๆ เป็นตัวอักษรอะไร โดยเราจะเอาค่าทั้ง 65 ค่านี้ใส่ไว้เป็น input ของ neural network เพื่อให้ neural network ทำการวิเคราะห์ว่า input ทั้งหมดที่ใส่เข้าไปนั้นจะให้ผลลัพธ์ออกมาเป็นตัวอักษรตัวอะไร ต่อไปเราจะพูดถึงกระบวนการทางด้าน neural network ซึ่งเป็นส่วนที่สำคัญสำหรับเอาไว้ recognition ตัวอักษร

3.6 ส่วนของ Neural Network ในโปรเจกต์ OCR เป็นดังต่อไปนี้

- ส่วนของการสร้าง network
- ส่วนของการ train data
- ส่วนของการ recognition

3.6.1 การสร้าง Neural Network

โครงสร้างของ neural network แบบ multilayer-layer ที่ใช้กันโดยทั่วไปและสามารถเอาไว้แก้ปัญหาได้เป็นจำนวนมากนั้น จะประกอบไปด้วย layer ทั้งสิ้นอยู่ 3 Layer ได้แก่ input layer, hidden layer และ output layer ใน project นี้ก็เช่นกัน neural network ที่ใช้ใน project นี้จะมีทั้งสิ้น 3 layer เช่นกันและการเชื่อมต่อในแต่ละ layer ก็จะเป็นแบบ fully-connected คือว่าทุกๆ node ในแต่ละ layer นั้นจะเชื่อมต่อถึงกันหมดเลยในการสร้าง neural network นั้นนอกจากที่เราจะกำหนดจำนวนของ layer ที่จะใช้แล้วยังต้องกำหนดจำนวน node ที่อยู่ในแต่ละ layer ด้วยซึ่งเราจะกำหนดจำนวน node ในแต่ละ layer เป็นดังต่อไปนี้คือ

- **input layer** จะมี node อยู่ทั้งสิ้นจำนวน 65 node ซึ่งก็จะหมายถึงว่าเราจะมี input ทั้งสิ้น 65 ตัวซึ่ง input ของ neural network ทั้งหมดก็คือ ตัวเลขทศนิยมที่แทน BLOB ตัวอักษรแต่ละตัวทั้ง 65 ค่า

- **hidden layer** ใน hidden layer นั้นไม่มีสูตรที่แน่นอนตายตัวที่จะกำหนดจำนวนของ node ที่จะอยู่ใน layer นี้ แต่ว่าโดยส่วนใหญ่แล้วมักจะกำหนดจำนวนของ hidden node ให้มีค่าอยู่ระหว่างจำนวน node ของ input layer และจำนวน node ของ input layer และจำนวน node ของ output layer เนื่องจากว่าเรากำหนดจำนวนของ hidden node เป็นอะไรก็ได้ใน project นี้จึงกำหนดให้มีค่าเป็น 65 เท่ากับจำนวนของ input node พอได้เลย

๘ DilleniaUPC

๘ EucrosiaUPC

๘ FreesiaUPC

๘ IrisUPC

เนื่องจากว่า project นี้เกี่ยวกับ OCR ดังนั้นภาพที่รับเข้ามาเพื่อจะทำการ recognition นั้นก็จะมาจาก Scanner (โดยการ scan เอาข้อความตัวอักษรเข้าไป ก็จะได้ไฟล์ภาพออกมา) ดังนั้น training data ที่เราใช้ทั้งหมดนี้จึงเป็น training data ที่เป็นภาพที่มาจากการ scan จริง ๆ ด้วยเช่นกันโดยไฟล์ภาพ training data ที่ได้มาทั้งหมดนี้ ถูก scan ที่ความละเอียด 300 dpi และใช้ชนิดของการ scan เป็นแบบ sharpen เพื่อให้มีความคมชัดมากขึ้น และภาพทุกภาพที่ใช้ล้วนแต่เป็นภาพที่ผ่านการ scan มาเพียงครั้งเดียวเท่านั้นด้วย ดังนั้นใน project นี้จึง recommend ภาพที่ผู้ใช้งานจะเอามาทำการ recognition ว่าควรจะเป็นดังที่กล่าวมาข้างต้นด้วย (สิ่งที่สำคัญก็คือ ความละเอียดของภาพนั้นไม่ควรที่จะต่ำกว่า 300 dpi ด้วย) นอกจากนี้ขนาดของ font ที่ใช้ใน training data นี้มีขนาดอยู่ตั้งแต่ 14-22 และ training data ที่ใช้ในครั้งนี้ก็จะมีทั้ง font ปกติ และ font ที่เป็นตัวเอียงด้วย

เมื่อมี training data แล้วขั้นตอนต่อไปคือการเอา training data ทั้งหมดมา train โดยใช้ neural network สำหรับการ train นี้ก็จะใช้ model ที่เรียกว่า back-propagation ในส่วนของการ train นั้นจะมีสิ่งที่แตกต่างจากส่วนของการ recognition คือว่า ในส่วนของการ train จะมีการกำหนด input และ กำหนด output ที่ถูกต้องแน่นอนให้กับ neural network จากนั้นการ train ก็จะเหมือนกับการปรับ weight ที่เชื่อมอยู่ระหว่าง node ทุก ๆ node ในแต่ละ layer ที่อยู่ใน neural network ให้มีค่าที่เหมาะสม (ตอนเริ่มต้นนั้นเรากำหนด weight โดยการ random ขึ้นมาก่อน) เพื่อที่จะทำให้ neural network นี้สามารถที่จะ recognition ได้อย่างถูกต้อง ในส่วนของการปรับ weight นั้นจะอาศัยวิธีการทาง neural network ที่เรียกกันว่า feed-forward ก่อนเพื่อที่จะคำนวณค่าของ error ของ weight จากนั้นเมื่อรู้ค่า error แล้วก็จะทำการ back-propagation หรือการแพร่กลับ เพื่อที่จะปรับ weight ให้มีค่า error ที่น้อยลง และในการ train ก็จะมีการทำซ้ำกระบวนการเช่นนี้ไปเรื่อย ๆ (คือทำ feed-forward ก่อนแล้วตามด้วย back-propagation) จนกระทั่งเราได้ค่า error ที่มีค่าน้อย ๆ ที่เป็นที่น่าพอใจ และมั่นใจพอว่า neural network ที่ผ่านการ train นี้สามารถที่จะ recognition ข้อมูลที่มันยังไม่รู้ได้แล้ว และสิ่งสุดท้ายที่เราทำก็คือ เราจะทำการ save weight เก็บไว้เพื่อเอาไว้ใช้งานในคราวต่อไป เพื่อที่เราจะได้ไม่ต้องมาทำการ train ใหม่อีกครั้ง เนื่องจากกระบวนการ train นี้เป็นกระบวนการที่ต้องอาศัยเวลาอย่างมาก ส่วน pseudo code ของการ train จะเป็นไปดังข้างล่างนี้

1. แปลงจาก training data ซึ่งเป็นรูปภาพ ให้อยู่ในรูปของ input vector ที่จำเป็นสำหรับตัวอักษรที่จะเอาไป train แต่ละตัว เพื่อเอาไปใส่เป็น input ของ neural network
2. กำหนด output ที่ถูกต้อง ซึ่ง match กับ feature ของตัวอักษรใด ๆ ที่ใส่เป็น input ของ neural network
3. ทำกระบวนการ feed-forward เพื่อที่จะคำนวณค่าของแต่ละ node ในทุก ๆ layer ในขณะนั้นว่าให้ผล

อะไรออกมา กระบวนการนี้จำเป็นเพื่อที่จะเอาค่าที่ได้จาก output node ของ neural network ไปเปรียบเทียบกับ output ที่เราอยากให้เป็นดังในขั้นตอนที่ 2 แล้วจะสามารถคำนวณค่า error ออกมาได้

4. คำนวณค่า error ออกมา จากนั้นก็เอาค่า error นี้บวกกับค่า error ที่ได้จากการ train ของแต่ละตัวอักษรในครั้งก่อน ๆ

5. ทำกระบวนการ back-propagation เพื่อเป็นการปรับ weight ที่เชื่อมในแต่ละ node ให้มันดีขึ้นเพื่อที่จะคำนวณค่าของ error ได้น้อยลงในครั้งต่อไป

6. ทำกระบวนการที่ 1 - 5 ใหม่แต่กับตัวอักษรตัวต่อไปที่อยู่ใน training data

7. เมื่อทำการ train จนครบทุก ๆ ตัวอักษรทั้งหมดของ training data แล้วให้หา error รวมโดยการเอา error ที่ได้จากการบวกกันไปเรื่อย ๆ ในขั้นตอนที่ 4 มาหารด้วยจำนวนรวมของตัวอักษรทั้งหมดที่อยู่ใน training data (รวมไปถึงตัวที่มันซ้ำกันแต่อยู่คนละไฟล์ด้วย)

8. ทำการ check ค่า error ที่ได้ในข้อ 7 กับค่าที่เราต้องการ หากพบว่า error มันต่ำกว่าค่าที่เรากำหนดไว้ก็ให้หยุดการ train ถือเป็นอันเสร็จสิ้นกระบวนการ train แต่ถ้าไม่ ก็ต้องกลับไปทำกระบวนการข้างต้นซ้ำอีกจนกว่าจะต่ำกว่าที่เรากำหนดไว้

3.6.3 การ recognition

เมื่อเรามีการ training data ไปแล้ว ส่วนต่อไปที่เราจะกล่าวถึงก็คือ การ recognition การ recognition คือการที่เราใส่ input ที่เป็น unknown เข้าไปแล้วให้ neural network ทำการวิเคราะห์ว่าสิ่งที่เราใส่เข้าไปในนั้นจะเป็นตัวอักษรอะไร เมื่อผู้ใช้ต้องการใช้งานโปรแกรมเพื่อ recognition ภาพ สิ่งที่เราทำก็คือ เราก็จะสร้าง neural network ขึ้นมาชุดหนึ่ง ซึ่งเป็นชุดที่มีลักษณะเหมือนกับตอนที่เรทำการ train data เมื่อสร้าง neural network เสร็จแล้วแทนที่เราจะทำการ train ใหม่ เราก็จะโหลด weight ที่ save เก็บไว้นั้นขึ้นมาแล้วเอาไปใส่ใน neural network แทน ในส่วนของการ recognition นั้นเราก็จะเอาข้อมูลที่ผู้ใช้ป้อนเข้ามา (ในที่นี้ก็ได้แก่ไฟล์ภาพจากเครื่อง scanner) มาทำการแปลงให้อยู่ในรูปแบบที่เหมาะสมสำหรับที่จะเอาเข้าสู่กระบวนการ recognition (ซึ่งกระบวนการแปลงต่าง ๆ นั้นได้กล่าวถึงมาก่อนหน้านี้แล้ว) หลังจากที่เราเอาข้อมูลมาแปลงเสร็จจนได้เป็นข้อมูลของแต่ละตัวอักษรไป เราก็จะเอาข้อมูลของแต่ละตัวอักษรนั้น (ในที่นี้ก็คือ feature ขนาด 65 อัน) ใส่เข้าไปเป็น input ของ neural network เพื่อที่จะให้ neural network นั้นทำการวิเคราะห์ออกมาว่าข้อมูลที่เรใส่เข้าไปจะเป็นตัวอักษรตัวอะไร ซึ่งเราจะใช้กระบวนการที่เรียกว่า feed-forward เพื่อหาค่าของ output node ที่มีค่าที่มากที่สุด และเราก็จะเอา output node ที่มีค่าที่สุคนั้น ไปตีความหมายเพื่อให้ได้ตัวอักษรจริง ๆ ที่สอดคล้องกับ input (การทำเช่นนี้ เรียกได้อีกอย่างว่า winner-take-all เพราะว่าเราจะเอา output ที่มีค่าที่สุดเป็นคำตอบ) และเราก็จะต้องทำกระบวนการนี้ซ้ำ ๆ กับทุก ๆ รูปตัวอักษรที่อยู่ในภาพ จนครบทุกตัวด้วย

3.7 Training set and validation set

เมื่อเรากำหนดข้อมูลใน Training data ทั้งหมดที่จะเอาไว้สำหรับเป็น training set และ validation set แล้ว ขั้นตอนต่อไปก็คือขั้นตอนของการ train ซึ่งในการ train ข้อมูลโดยใช้ training set และ validation set นั้นจะมีความแตกต่างจากขั้นตอนการ train โดยทั่วไป ดังนี้คือ

- ในขั้นตอนของการ train นั้น จะใช้ข้อมูลจากส่วนของ training set ซึ่งมีอยู่ 50% ของข้อมูลทั้งหมดนั้น เราจะเอาข้อมูลใน training set มา train ก่อน โดยจะมีการกำหนดจำนวน epochs ที่ต้องการที่แน่นอนไว้ด้วย เช่น 500 รอบ, 1000 รอบ, 1500 รอบ เป็นต้น

- ขั้นตอนต่อไป หลังจากที่เราทำการ train ข้อมูลไปจนครบจำนวนของ epochs ที่เรากำหนดไว้ตอนต้นแล้ว

ต่อไปเราก็จะเอา neural network ที่ผ่านการ train ไปทำการทดสอบกับ validation set การทดสอบกับ validation set ในที่นี้ก็คือการตรวจสอบความถูกต้องของ โครงข่าย neural network ที่เราได้ train มาแล้ว โดยเราจะเอาข้อมูลของแต่ละตัวอักษรใน validation set มาใส่ไว้เป็น input ของ neural network แล้วเราก็จะกำหนด output ที่ถูกต้องซึ่ง match กับข้อมูล input ที่เราได้ใส่เป็น input จากนั้นเราก็จะใช้กระบวนการ feed-forward ของ neural network กับข้อมูล input นั้น แล้วดูว่ามันให้ผลลัพธ์ออกมาเป็นตัวอะไร ซึ่งถ้ามันเหมือนกับข้อมูลของ output ที่ซึ่งเราได้กำหนดไว้ตั้งแต่ตอนแรกนั้น เราก็จะถือว่ามัน match กัน ถือว่าถูกต้อง

แต่ถ้าหากว่าให้ output ออกมาแล้วไม่ match กับข้อมูล output ที่ซึ่งเราได้กำหนดไว้ตั้งแต่ตอนต้น ก็ถือว่ามันไม่ match กัน และเราก็จะทำแบบนี้กับข้อมูลตัวอักษรทั้งหมดที่อยู่ใน validation set แล้ววัดเป็นเปอร์เซ็นต์ของความถูกต้องออกมาโดยดูว่า neural network มันให้ผลลัพธ์ออกมาถูกต้องกี่เปอร์เซ็นต์ จากนั้นก็เอาเปอร์เซ็นต์ความถูกต้องได้นี้มาเปรียบเทียบกับค่าที่เราได้กำหนดไว้ค่าหนึ่ง ซึ่งถ้าเปอร์เซ็นต์ความถูกต้องที่วัดได้มีค่าที่มากกว่าหรือเท่ากับที่เราได้กำหนดไว้เนี่ย เราก็จะหยุดการ train เพียงแค่นั้น และถือว่าข้อมูลมีความถูกต้องในระดับหนึ่งแล้ว และจะหยุดกระบวนการ train ไว้เพียงเท่านี้ด้วย แต่ถ้าหากว่าเปอร์เซ็นต์ความถูกต้องที่วัดได้มีค่าน้อยกว่าค่าที่กำหนดไว้ เราจะต้องกลับไปทำการ train ต่อ เพราะถือว่าข้อมูลยังมีความถูกต้องไม่เพียงพอ เราก็จะกลับไป train เท่ากับจำนวน epochs ที่เราได้กำหนดไว้ตั้งแต่ตอนต้น เมื่อ train เสร็จก็เอาไปทดสอบกับ validation set ต่อ และก็จะทำแบบนี้ไปเรื่อยๆจนกว่าเราจะได้เปอร์เซ็นต์ความถูกต้องที่มากกว่าค่าที่เราได้กำหนดไว้จึงจะหยุด

ปัญหาหนึ่งที่อาจจะพบได้ในการ train แบบนี้คือ เมื่อเราทำการ train ไปเรื่อยๆ กลับจะพบว่ามีการแกว่งของค่าเปอร์เซ็นต์ความถูกต้องที่วัดได้จาก validation set นั่นคือ เราสังเกตได้ว่าเมื่อเราทำการ train ไปเรื่อยๆ แล้วเอาไปทดสอบกับ validation set เปอร์เซ็นต์ความถูกต้องที่ได้ก็จะมีค่าที่เพิ่มมากขึ้นเรื่อยๆ แต่บางครั้งเราพบว่า เปอร์เซ็นต์ความถูกต้องของครั้งใหม่นั้น มีค่าน้อยกว่าเปอร์เซ็นต์ความถูกต้องของครั้งเก่า เมื่อเป็นเช่นนี้ก็แสดงได้เลยว่า neural network เริ่ม overfit แล้ว เราก็จะทำการแก้ไขโดยการ backtrack กลับไปที่ state ก่อนหน้าก่อนที่เปอร์เซ็นต์ของความถูกต้องจะลดลง ซึ่งคือ state ที่ให้ค่าของเปอร์เซ็นต์ความถูกต้อง

ออกมาสูงที่สุด จากนั้นก็โหลดค่าของ weight เดิมที่เราได้ทำการ backup เก็บไว้ที่ state นั้นกลับเข้ามาใส่ให้แก่ neural network และก็ลดจำนวนของ epochs ลงให้กลับไปมีค่าเท่ากับค่าของ epochs ตอนที่อยู่ที่ state ก่อนหน้าด้วย เมื่อเรา backtrack กลับไปก่อนหน้าแล้วสิ่งที่เราทำต่อไปก็คือ ลดค่าของ learning rate ลง ทั้งนี้เพื่อให้อัตราของการเรียนรู้ที่น้อยลง หรือว่าช้าลงด้วย ซึ่งจะช่วยให้ไม่เกิดการ oscillate ขึ้นอีก นอกจากนี้หลังจากที่เราลดค่าของ learning rate ลงเสร็จแล้ว เราก็อาจจะลดค่าของ epochs ลงได้ด้วยเช่นกัน ทั้งนี้ เราลดค่าของ epochs ลงก็เพื่อให้เราสามารถนำ neural network นั้นไปเข้าสู่กระบวนการตรวจสอบโดยใช้ validation set ได้บ่อยขึ้น

3.8 Algorithm ในการเรียงลำดับตัวอักษร

หลังจากที่เราหาระดับต่างๆในแต่ละบรรทัดได้แล้ว (ในที่นี้ก็คือระดับตั้งแต่ระดับที่ 0 ซึ่งก็คือระดับของสระอู สระอุ เป็นต้น ไปจนถึงระดับบนสุด ซึ่งก็คือระดับที่ 3) รวมทั้ง blob ตัวอักษรแต่ละตัวที่เราสามารถแยกออกมาจากรูปภาพได้ ก็ผ่านขั้นตอนของการ Recognition จนได้มาเป็นตัวอักษรจริงๆซึ่ง match กับ blob ตัวนั้นๆแล้วทุกๆตัว ขั้นตอนต่อไปก็คือขั้นตอนของการเรียงตัวอักษรต่างๆที่ recognize ออกมาได้นั้นให้เป็นข้อความที่ถูกต้อง เหมือนกับไฟล์รูปภาพก่อนกระบวนการ process ต่างๆ ซึ่งการเรียงตัวอักษรต่างๆที่ recognize ออกมาได้นั้นให้เป็นข้อความที่ถูกต้องนั้น เราจะอาศัยการดูจากตำแหน่งของ blob แต่ละตัวเป็นหลัก ทั้งนี้ก็เพราะว่าลำดับของ blob แต่ละตัวที่ออกมาได้นั้น มักจะเป็นลำดับของการพิมพ์ที่ไม่ถูกต้อง ลองดูจากรูปข้างล่างนี้

เขาอยู่ที่นั่น

รูปที่ 3.17 ตัวอย่างของข้อความที่ต้องผ่านการเรียงลำดับตัวอักษรใหม่

จากรูปข้างบนนี้เมื่อเราสามารถแยก blob ตัวอักษรแต่ละตัวออกมาได้ ลำดับของ blob ที่สามารถแยกออกมาได้จะเป็นดังนี้คือ อันดับ 1 ได้แก่ ไม้เอกที่อยู่ในระดับที่ 3 อันดับ 2 ได้แก่ ไม้โทที่อยู่ในระดับที่ 3 อันดับ 3 ได้แก่ ไม้เอก ที่อยู่ในระดับที่ 2 อันดับ 4 ได้แก่ สระอี ที่อยู่บนตัว ท เป็นต้น ที่เป็นเช่นนี้ก็เนื่องจากว่า เราได้ทำการ process ภาพเพื่อหา blob จากบนลงมาสู่ล่าง และก็จากซ้ายไปขวาตามลำดับ จึงได้เจอ sequence ของ blob เป็นดังที่กล่าวมา เมื่อเป็นเช่นนี้จึงทำให้เราต้องทำการเรียงลำดับของ blob ใหม่ เพราะว่าตอนที่เรทำการพิมพ์ข้อความจริงนั้นเราไม่สามารถพิมพ์ไม้เอกมาก่อนแล้วตามด้วยไม้โทดัง sequence ข้างบนได้ เราจะต้องพิมพ์ blob ที่อยู่ในระดับที่ 1 ก่อน ซึ่งในที่นี้ก็คือได้แก่ สระเอ (อ) คือตัวแรกที่เราต้องพิมพ์ลงไป ใน file ผลลัพธ์ จากนั้นก็ตามด้วย ข,ว ตามลำดับ พูดย่อยก็คือ พิมพ์ตัวอักษรในระดับที่ 1

เรียงตามลำดับจากซ้ายไปขวา และถ้าหากว่าตัวอักษรที่เราพิมพ์ซึ่งอยู่ในระดับที่ 1 นั้นมีวรรณยุกต์หรือสระอื่นๆประกอบอยู่ข้างบนหรือข้างล่างของตัวมัน

เราก็จะพิมพ์สระหรือวรรณยุกต์ตัวนั้นๆต่อจากพยัญชนะ โดยมีข้อแม้ว่าเมื่อพิมพ์พยัญชนะเสร็จแล้วจะทำการพิมพ์สระหรือวรรณยุกต์ ถ้าหากว่าข้างใต้พยัญชนะมี สระอุ หรือ สระอู อยู่ และมีวรรณยุกต์ที่อยู่บนพยัญชนะอยู่ด้วย เราจะต้องพิมพ์ระดับที่ 0 ก่อน ซึ่งในที่นี้ก็ได้แก่ ว่า เราจะต้องพิมพ์ สระอุ หรือ สระอู ก่อน แล้วจึงตามด้วยวรรณยุกต์ที่อยู่บนพยัญชนะตัวนั้นๆตามมา แต่ถ้าหากมีสระหรือวรรณยุกต์ที่อยู่ในลำดับที่ติดกัน เช่น มีสระอยู่ในระดับที่ 2 และมีวรรณยุกต์อยู่ในระดับที่ 3 ถ้าเป็นเช่นนี้เราก็จะทำการพิมพ์สระที่อยู่ในระดับที่ 2 แล้วตามด้วยวรรณยุกต์ที่อยู่ในระดับที่ 3 ตามลำดับ แต่ถ้าหากว่า พยัญชนะนั้นๆมีสระหรือวรรณยุกต์กำกับเพียงตัวเดียว เราก็ทำการพิมพ์ตัวนั้นต่อจากพยัญชนะได้เลยโดยไม่มีข้อแม้ใดๆเลย

Flowchart ของอัลกอริทึมแบบ Training Set and Validation Set ดังนี้

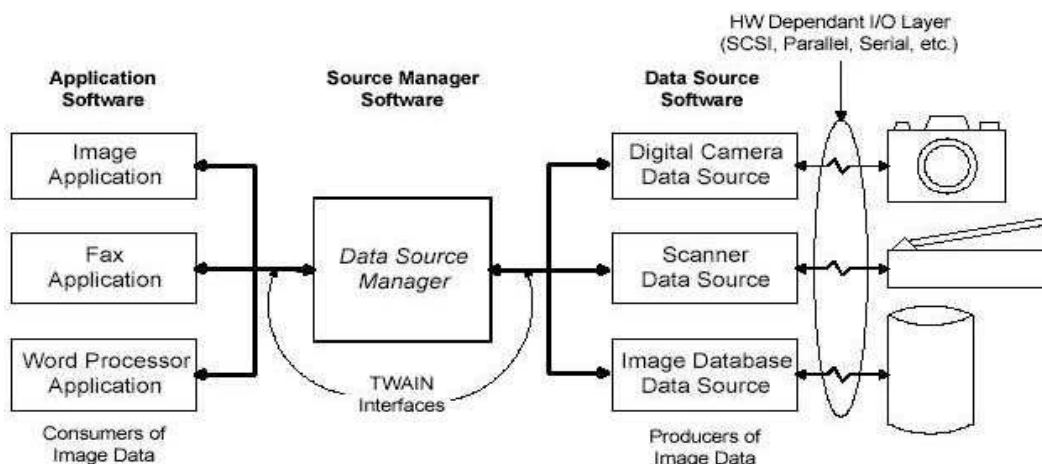


3.9 การ Interfacing กับ scanner

ในโปรเจกต์นี้อนุญาตให้มีการรับภาพมาจาก scanner ได้ ซึ่งผู้ใช้งานสามารถรับภาพจาก scanner ได้จากภายในตัวโปรแกรมได้เลย ซึ่งจะมีปุ่มสำหรับการเลือกชนิดของ scanner ที่ติดต่อกับคอมพิวเตอร์ และก็มีปุ่ม scan ซึ่งเอาไว้สำหรับ scan เพื่อให้ได้ภาพออกมา สำหรับในโปรแกรมที่พัฒนาขึ้นมานั้น เมื่อ scan ภาพเข้ามาจาก scanner แล้ว ภาพจะมีความละเอียด 300 dpi ซึ่งเป็นความละเอียดที่ค่อนข้างจะสูง แต่มีข้อดีคือจะทำให้โปรแกรมสามารถ process ภาพได้ดีมากยิ่งขึ้น รวมทั้งมี noise ที่ไม่มากด้วย สำหรับการเขียนโปรแกรมเพื่อติดต่อกับ scanner นั้น เราจะอาศัย tool หนึ่งที่ชื่อว่า twain ร่วมกับ library ของ EZTWAIN ทั้งนี้เนื่องจากว่าตัว twain นั้นมีโครงสร้างที่ซับซ้อนและยาก และต้องใช้ระยะเวลานานกว่าจะศึกษาทำความเข้าใจถึงฟังก์ชันการคำนวณของมันทั้งหมด จึงเลือกใช้ EZTWAIN ซึ่งเป็น library สำหรับไปติดต่อกับ twain อีกที ทั้งนี้เพราะ EZTWAIN นั้น ได้ encapsulate การติดต่อกับ twain ในส่วนที่ค่อนข้างจะยุ่งยากไว้หมดแล้ว เหลือเพียงให้เราเขียน code เพื่อ interface กับตัวของ EZTWAIN เอง ซึ่งมีความง่ายกว่าการเขียน code เพื่อ interface โดยตรงกับ twain มากๆ ต่อไปจะพูดถึง twain และโครงสร้างของ twain

Twain คือเป็นการริเริ่มจัดทำมาตรฐานการติดต่อสื่อสาร (โปรโตคอล) ที่ใช้กับอุปกรณ์ประดิษฐ์เกี่ยวกับภาพ, รูป (Imaging Devices) เกิดขึ้นจากกลุ่มอุตสาหกรรมผู้ผลิตอุปกรณ์รวมตัวกันโดยไม่หวังผลประโยชน์ ซึ่งต้องการมาตรฐานซอฟต์แวร์ในการติดต่อสื่อสารและเครื่องมือในการเขียนโปรแกรมเพื่อติดต่อกับอุปกรณ์ (API: Applications Programming Interface) ซึ่งตั้งข้อกำหนดในการใช้งานขึ้นมา 3 ส่วนประกอบหลักของ Twain คือ

1. Application Software
2. Source Manager Software
3. Data Source Software



รูปที่ 3.18 รูปแสดงไดอะแกรมการติดต่อระหว่างคอมพิวเตอร์กับอุปกรณ์ที่ใช้ TWAIN

ต่อไปจะกล่าวถึง EZWAIN และฟังก์ชันต่างๆที่จำเป็นต้องเรียกใช้เพื่อให้สามารถรับภาพเข้ามาจาก scanner ได้ฟังก์ชันต่าง ๆ ของการรับภาพจาก Scanner โดย Ezwain มีหน้าที่ดังนี้

Twain_SelectImageSource

ฟังก์ชันนี้มีไว้สำหรับเลือก Twain device (ในที่นี้ก็ได้แก่เครื่อง scan) ที่ต่อเชื่อมอยู่กับเครื่องคอมพิวเตอร์ของเรา ในกรณีที่มี device เพียงอันเดียวที่เชื่อมต่ออยู่ ก็ไม่จำเป็นต้องเรียกฟังก์ชันนี้ก็ได้ ซึ่งฟังก์ชันนี้จะทำงานก็ต่อเมื่อมี device มากกว่า 1 ตัว ให้เราเลือก

Twain_SetHideUI

ฟังก์ชันนี้จะทำหน้าที่ซ่อนหน้าต่างต่างของ User interface ไม่ให้ผู้ใช้เห็น

Twain_OpenDefaultSource

ฟังก์ชันนี้อาจใช้สำหรับตรวจสอบว่า Dialog ของ source select นั้นเคยเปิดมาก่อนแล้วหรือยัง ถ้าหากว่าเคยเปิดมาก่อนแล้ว ก็จะไม่ทำอะไรเลย และก็จะ return ค่าออกมาเป็น true และจะ return ค่าเป็น false ถ้าหากว่า source manager นั้นไม่ถูก load และไม่ถูกเปิด

Twain_SetCurrentUnits

ฟังก์ชันนี้ใช้สำหรับการเซตหน่วยการวัดในปัจจุบันสำหรับ Source หน่วยของการวัดนี้จะถูกประกาศไว้ในไฟล์ Twain.h ในที่นี้เรากำหนดให้หน่วยของการวัดอยู่ในหน่วยของ นิ้ว

Twain_SetCurrentPixelFormat

ฟังก์ชันนี้ทำการเซตรูปแบบปัจจุบันของจุด เพื่อที่จะรับเข้ามาจาก Scanner ให้ได้ไปเป็นตามรูปแบบนั้น

Twain_SetBitDepth

ฟังก์ชันนี้จะทำการเซตความลึกของบิต ที่เอาไว้สำหรับแสดงค่าสีของ Pixel ใด ๆ ในที่นี้เซตเป็น 24 บิต หมายความว่า 1 pixel นั้นใช้ 24 บิตในการเก็บ แบ่งเป็น Red 8 บิต Green 8 บิต และ Blue 8 บิต

Twain_SetCurrentResolution

ฟังก์ชันนี้ทำการเซตความละเอียดของภาพ ที่เราต้องการจะรับมาจาก Scanner ความละเอียด จะอยู่ในรูปของจำนวนจุดต่อหน่วยปัจจุบัน ซึ่งหน่วยปัจจุบันนั้น เราได้เซตไว้โดยฟังก์ชัน Twain_SetCurrentUnits เรียบร้อยแล้ว

Twain_AcquireNative

ฟังก์ชันนี้จะทำการรับรูปภาพเข้ามาจาก Data source ที่ถูกเลือกอยู่ในปัจจุบันจำนวนเพียงแค่ 1 รูปภาพเท่านั้น ฟังก์ชันนี้จะ return ค่าส่งกลับเป็น handle DIB ของรูปภาพที่ถูกรับเข้ามา

Twain_WriteNativeToFilename

ฟังก์ชันนี้จะทำหน้าที่เขียนไฟล์ภาพซึ่งเราได้รับมาจาก Scanner โดยภาพที่เรารับเข้ามาจาก scanner นั้นจะถูกเก็บไว้เป็น handle DIB ที่ได้มาจากฟังก์ชัน Twain_AcquireNative แล้วจากนั้นเราก็จะต้องส่ง handle DIB ตัวนี้ไปเป็นพารามิเตอร์ของฟังก์ชันนี้ด้วย เพื่อให้มันสร้างไฟล์ภาพที่เราต้องการออกมาให้

